Exact quotes, private contexts and reconstruction risk

Effectiveness report | 19 October 2026 | REA 6.7.0 | 24 fresh Astra/max agents

What the experiment found

Keeping genuinely private customer pricing rules behind consistent server permissions reduced other-context within-1% reconstruction from 100.0% to 0.0% in this study, a 100.0 pp mean reduction. Exact legitimate workflow actions passed 1,008/1,008. The same agents still reconstructed their own permitted pricing within 1% on 100.0% of heldouts. This protects private context; it does not stop copying the behaviour a customer can already use.

The bound-context design passed the predeclared useful private-context protection screen on all six targets. The screen required at least a 20 percentage-point reduction in other-context within-1% accuracy on every target, valid predictors for all six bound agents, zero other-context numeric disclosures and 100% exact workflow action success. No application rounding or query cap was used. Each legitimate customer could create a new exact quote for any integer quantity from 1 to 1000. This addresses the earlier studies' restriction to two pre-approved exact quantities.

Effectiveness scorecard

Policy	Own within 1%	Other within 1%	Other within 5%	Exact workflow actions
Client-selected context (unsafe)	100.0%	100.0%	100.0%	1,008/1,008
Bound private contexts	100.0%	0.0%	4.0%	1,008/1,008
Bound context + leaky export	100.0%	100.0%	100.0%	1,008/1,008
Bound context + shared rule	100.0%	100.0%	100.0%	1,008/1,008

Valid predictors: 24/24; failures: 0. Scores are means across six paired targets, with 128 common unseen quantities per target. Within 1% means absolute error no greater than 1% of the true exact price. A low other-context score is not a whole-SaaS cloning score. Exact workflow actions include queue creation, which has no price payload; every price-bearing response was checked in pence.

Policy	Own exact	Other exact	Other within-1% reduction
Client-selected context (unsafe)	100.0%	100.0%	0.0 pp
Bound private contexts	100.0%	0.0%	100.0 pp
Bound context + leaky export	100.0%	100.0%	0.0 pp
Bound context + shared rule	100.0%	100.0%	0.0 pp

Reductions are percentage points relative to the client-selected-context baseline. Shared-rule is a structural negative control: its own function is deliberately changed to match other. Its other target remains the same, but its own score is not a paired comparison on identical own truth.

What each policy actually changes

- Client-selected context: the server trusts a caller's requested customer context. Both independently parameterised pricing functions become queryable. This deliberately unsafe baseline tests whether the agents can reconstruct accessible rules.

- Bound private contexts: derive entitlement from the authenticated customer and apply it to create, alias, get, revise, accept, export, job creation and job retrieval. Return exact, minimal quote records. The other customer has separately provisioned private pricing parameters. Own queries are unlimited by the application.

- Leaky export: keep quote/object checks, but allow a convenience export to calculate any context and quantity. This deliberately weakens one route. Its other-context within-1% accuracy was 100.0%.

- Shared rule: keep all permission checks, but use the identical pricing function for both customers. An agent can transfer its own reconstruction without reading the other context. Other-context within-1% accuracy was 100.0%.

Agents were told the policy, the convenience-export weakness where present, the shared-rule relationship where present, and known other-object IDs before probing. These are informed failure controls, not blind discovery-rate estimates. These are tests of consistent authorisation plus genuinely private context parameters. Minimal responses, strict fields, version checks and quote snapshots are components of the protected design; this study does not estimate a separate accuracy reduction for each component. Do not invent customer-specific prices solely to create obscurity. Apply separation where real contracts, configurations or customer data are already private.

Paired results: every target

Target	Unsafe other within 1%	Bound other within 1%	Reduction
target-01	100.0%	0.0%	100.0 pp
target-02	100.0%	0.0%	100.0 pp
target-03	100.0%	0.0%	100.0 pp
target-04	100.0%	0.0%	100.0 pp
target-05	100.0%	0.0%	100.0 pp
target-06	100.0%	0.0%	100.0 pp

Targets 01-02 use volume pricing, 03-04 graduated pricing and 05-06 minimum/rebate pricing. Own and other parameters are independently sampled conditional on family. Agents are not told those families or parameter ranges. Six targets and one agent per cell support these observed paired differences, not a population confidence interval or a universal guarantee.

Exact customer workflows remained useful

All four policies were tested with both separately authenticated customers on every target: 48 HTTP journeys and 4,032 actions. Each customer created 12 fresh quotes at quantities 31, 67, 83, 119, 163, 211, 277, 349, 431, 557, 683 and 827. Each quote was revised by +13, accepted, retrieved, exported, queued for background export and retrieved from the job. Own-context journeys switched between two sessions of the same customer. Other-context utility used that other customer's real synthetic entitlement, which was never supplied to the reconstruction agent.

There were 3,456 price-bearing responses, of which 3,456 passed the exact-price and record checks. The utility pass condition was 100% successful actions, correct quote context, quantity, version, state and exact price wherever returned.

There was no privileged good-user exemption, no two-item catalogue, no manual approval delay and no rounding. Protected exports require an existing quoteId. A standalone quantity-to-export request is refused even in own context; the customer must create a quote first. This workflow constraint is not separately scored as an allowed convenience-export task, and product teams must assess that cost.

Bound private contexts passed 12/12 journeys. All policies together passed 4,032/4,032 actions.

For a concrete retained journey on target-01, the customer created quantity 31 at GBP 953.56, revised it to quantity 44 at GBP 1353.44, accepted version 2, then received that exact amount again from direct export and the background job. This is an observed new quote, not a fixed approval catalogue.

These checks establish synthetic workflow correctness, not human usability, real invoicing compliance, throughput or production latency. A product that legitimately offers cross-customer comparisons needs a different entitlement definition.

Hard blocks and the two-world check

Policy	Other numeric responses	Other distinct target/quantity pairs
Client-selected context (unsafe)	576	576
Bound private contexts	0	0
Bound context + leaky export	569	569
Bound context + shared rule	0	0

The bound agents received 0 other-context numeric disclosures. Known other quote and job IDs were supplied in advance, so identifier secrecy cannot explain protection. The separate deterministic check kept own rules fixed and changed hidden other rules. It compared 24,108 paired status/body responses (48,216 engine evaluations), including all 1000 quantities for own creation and other creation, alias and convenience export, plus objects, revisions, acceptance, jobs and revocation. All 24,108 paired responses matched exactly. These are in-process engine traces, not additional REA or HTTP requests. The 18 deliberate selector/export/shared-rule controls exposed or reproduced the protected exact value.

This demonstrates functional response independence for those traces. The source also explains the mechanism: in the bound arm, permitted own computations never read the other pricing parameters, and other-object requests receive constant denials. Changing hidden rules therefore cannot teach the caller those changes through these response bodies. Timing, resource use, transport metadata, caches, arbitrary vulnerabilities and future routes are excluded. The teaching fixture uses global sequential IDs, which could reveal activity counts if customers operate concurrently; concurrent activity confidentiality was not tested. Use opaque or appropriately scoped production IDs while retaining object permission checks. A zero-information statement would need those channels and the prior knowledge model specified; this report does not claim one for a production SaaS.

Stronger informed-prior sensitivity

All 6/6 bound-context agents submitted a copy of their correctly reconstructed own model as an explicitly uncertain fallback for other. The 0% within-1% result is the measured performance of those submitted guesses, not proof that every possible guess must fail. Their notes acknowledge the absence of other-context price observations.

A supplementary numerical predictor was given the correct target family and all historical parameter ranges, more prior knowledge than the agents. At each quantity it chose the integer price covering the largest within-1% interval among 4096 independently drawn prior prices. It used no target coefficients or other-context observations. Its code was hash-frozen during the agent run before any target disclosure or scoring; it was not part of the original primary protocol.

This stronger prior-only predictor achieved 1.7% within 1% and 12.4% within 5% on the same unseen inputs. It is a 1000-value numerical lookup, not another REA-assisted agent and not subject to the agent's 32-segment limit. Its purpose is to expose the effect of informative prior knowledge. It does not prove an optimal attacker bound, and agents with real public pricing datasets or colluding customer accounts may do better. Per-target predictions are retained in informed-prior.json.

How REA and the agent obtained the evidence

The LLM inspected delivered HTML and JavaScript, selected workflow JSON requests and asked the wrapper to fill the real browser UI. Pinned REA 6.7.0 capture-browser-scenario launched headless Chrome, executed fill/click/wait actions, and retained DOM and network bodies. The agent observed exact quotes, refusals and object responses, then used a separate generic numerical fitter on its successful captured prices. It selected new probes to narrow transition gaps and returned two bounded numerical predictors. REA collected evidence; the LLM chose the experiment and reconstruction. No server source was delivered. The captured business calls were 3,591 create, 1,005 export, 5 get, 7 job-read. 24/24 agents used the fitter. The full quote lifecycle was exercised separately by HTTP utility tests; a route offered to an agent is not automatically a route it tested through REA.

There were 338 real REA operations, 4,608 matched business responses, 9,892 checked captured-body hashes, 88 numerical fitting calls and 6,144 held-out predictions. REA completeness statuses: 338 incomplete. Overall incomplete capture is not treated as protection: every scored DOM row was matched to a distinct captured POST request and response, and every captured body's bytes and SHA-256 were checked.

Policy	Requests / agent	REA captures / agent	Minutes / agent
Client-selected context (unsafe)	192.0	14.8	5.5
Bound private contexts	192.0	13.0	5.1
Bound context + leaky export	192.0	14.5	6.2
Bound context + shared rule	192.0	14.0	5.1

The maximum per agent was 192 attempted requests including refusals, 64 REA captures, 40 fitting calls and 30 minutes. This is a research resource allowance, not a product defence. Agents could stop early. Recorded cumulative token usage was 5,461,828 across final checkpoints; cached/repeated context is included and no money estimate is inferred. Two private REA smoke captures validated the harness before the study and are excluded from measured operation totals. No score-selected reruns were used.

Full-domain diagnostic

Policy	Own within 1%	Other within 1%	Other within 5%
Client-selected context (unsafe)	100.0%	100.0%	100.0%
Bound private contexts	100.0%	0.0%	4.2%
Bound context + leaky export	100.0%	100.0%	100.0%
Bound context + shared rule	100.0%	100.0%	100.0%

This secondary score covers all 1000 quantities per context and includes observed inputs. The primary score uses only common unseen inputs chosen after all predictors were frozen. It should not replace the primary result.

Evidence integrity and laptop privacy

Before generation, the runner hashed the plan, prompt, target generator, fixture, tools, numerical fitter, scorer and utility checks. It committed hidden targets by SHA-256, randomised the 24-cell schedule, and isolated each fresh agent in an empty read-only context with no network or host environment access. All predictor files were frozen before hidden targets and holdouts were published. Private raw app-server/account events are excluded. The configured model service processed only the synthetic task observations supplied by the harness. This is not a VM isolation or comprehensive laptop security audit.

The offline verifier replays every captured workflow against the fixture, each fitting call using only observations available at that moment, predictor/target hashes, all scores, HTTP utility and deterministic two-world traces. Public evidence contains generic loopback fixture addresses, not the operator's network addresses. The publication process scans PDFs, archives, decoded bodies and Git identity before pushing. Actual release checks are recorded separately in docs/CONTEXT_RELEASE_CHECKS.md.

Practical interpretation

The bound-context design passed the predeclared useful private-context protection screen on all six targets. The feasible improvement is to keep exact customer work while preventing one account from querying everyone's private rules. A leaky alternate path or a shared global function can erase that benefit. Own-context behaviour, delivered UI and generic workflows remain observable and potentially copyable. The study has not made an arbitrary SaaS product reconstruction-proof.

Reproduce and investigate

Read the frozen protocol, participant prompt, runner guide and raw evidence. From the full repository, run node context-study/verify.mjs and node --test context-study/test.mjs. See how REA works for pinned source references.

The permission design follows the problem described in OWASP API1:2023. Browser capabilities are documented by REA. The restricted agent loop uses the official Codex app-server protocol. These sources explain mechanisms; measured percentages come from this repository's retained evidence.